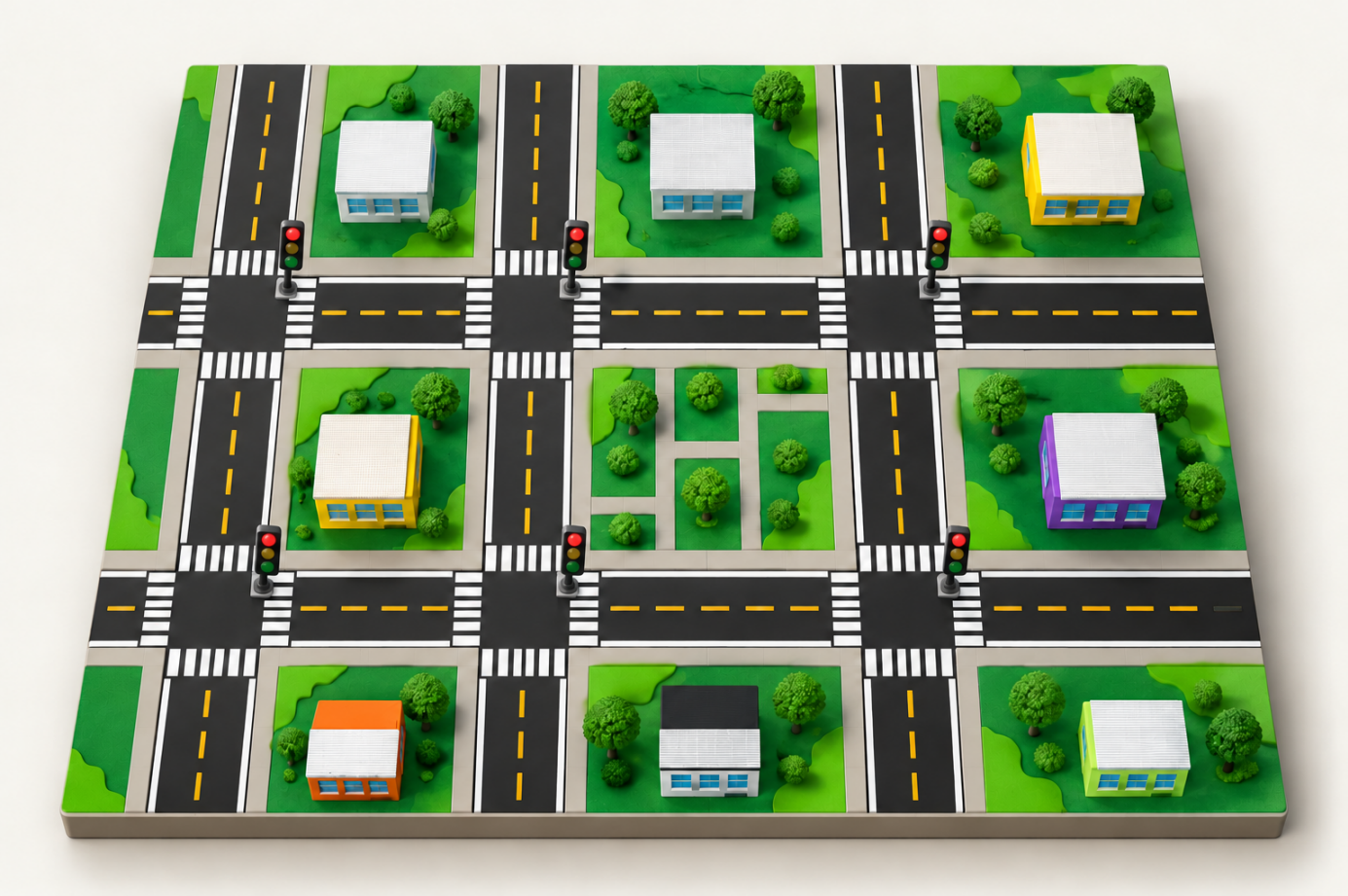
SMARTTRAFFIC · GUÍA DIDÁCTICA DE ELECTRÓNICA

Seis semáforos con Arduino UNO R3

Cableado soldado paso a paso · 18 LEDs individuales · comunicación USB



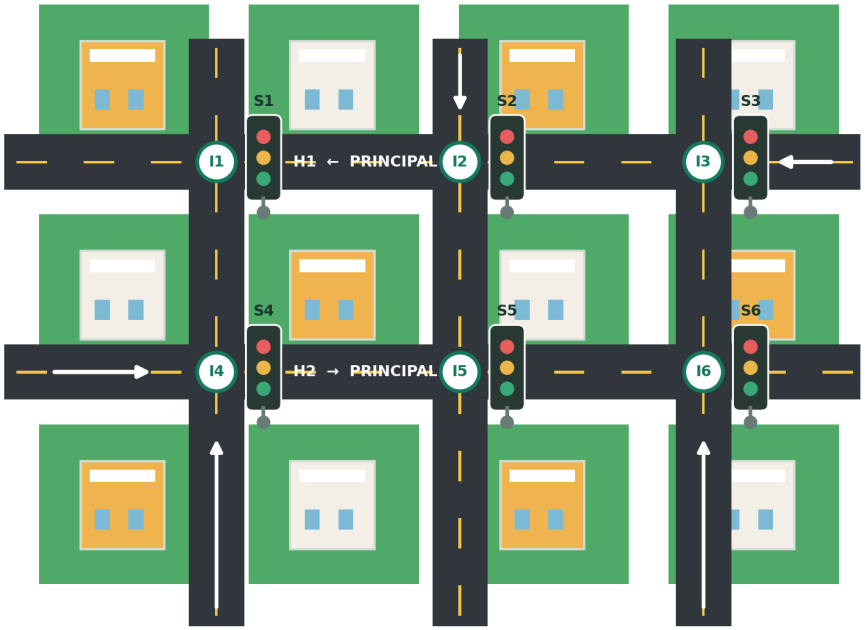
Circuito implementado en el repositorio

18 salidas directas · 18 resistencias de 2,2 kΩ · GND común
Sin protoboard, sin 74HC595 y sin transistores para LEDs indicadores
El dibujo de portada es ilustrativo; las páginas siguientes fijan cada pin y retorno.

Qué vas a construir

El proyecto conecta la maqueta física, el Arduino y el panel web.

Vista general · 6 cruces y 6 semáforos



ARDUINO UNO R3
USB a la computadora
18 señales + GND común

S1–S6 → I1–I6 (propuesta)
Cada semáforo mira a una horizontal.
Las verticales llevan ceda el paso.

Esquema aproximado de la maqueta; no representa distancias ni la ruta física del mazo de cables.

- Cada semáforo usa tres LEDs individuales: rojo, amarillo y verde.
- Un cable por ánodo lleva la señal desde un pin; los tres cátodos regresan por resistencias separadas a un GND común.
- La placa se alimenta por USB. La aplicación lee su puerto mediante un puente serial local en la misma computadora.
- Ruta general propuesta: llevar cada mazo por debajo de la base y por el borde hasta el Arduino. La longitud y fijación dependen de tu maqueta.

Asignación física propuesta (debe verificarse al instalar)

S1→I1, S2→I2, S3→I3, S4→I4, S5→I5 y S6→I6. Los seis controlan las vías horizontales principales; las verticales llevan ceda el paso.

Lista de piezas y herramientas

Las cantidades son para seis semáforos de tres LEDs cada uno.

Cant.	Material	Uso
1	Arduino UNO R3	Seis digitales D2–D13 y seis analógicos A0–A5 usados como salidas.
6	LED rojo individual, 2 patas	Uno por semáforo; comprobar polaridad y hoja de datos.
6	LED amarillo individual, 2 patas	Uno por semáforo; no usar módulo RGB común.
6	LED verde individual, 2 patas	Uno por semáforo; brillo depende de Vf.
18	Resistencia 2,2 kΩ, ¼ W	Una resistencia en serie por cada LED.
18 + bus	Cable flexible de señal y cable negro de retorno	Rojo, amarillo y verde por S1–S6; negro para GND común. Etiquetas en ambos extremos.
1	Cable USB de datos compatible	Alimenta el UNO y ofrece comunicación serial; no usar cable solo de carga.
1	Condensador cerámico 100 nF (104)	Recomendado cerca del mazo entre 5V y GND; no tiene polaridad.
1	Cautín, soporte y estaño	Trabaja ventilado y sobre superficie resistente al calor.
Varios	Termorretráctil, cortacables, etiquetas y multímetro	Aislar soldaduras, recortar patas, marcar cables y revisar continuidad.

Piezas que no lleva este circuito

El firmware no usa 74HC595, transistores ni una fuente externa para LEDs indicadores. No compres esos componentes para este cableado.

Por qué el UNO alcanza

La tabla del firmware y el circuito descrito en ELECTRONICA.md coinciden.

- 6 semáforos × 3 luces = 18 salidas. El UNO tiene 20 pines que pueden actuar como digitales; D0/RX y D1/TX se reservan para USB serial.
- Quedan exactamente D2–D13 (12 pines) y A0–A5 (6 pines). El programa los configura como OUTPUT; no se deja una salida física libre.
- Cada pin alimenta solo un LED indicador de baja corriente a través de su resistencia. El retorno termina en GND del mismo UNO.
- Los 74HC595 no forman parte del montaje final. Harían falta si se añadieran luces físicas, pero cambiarían cableado y firmware.
- El LED de D13 puede hacer que también se encienda el indicador integrado de la placa cuando S4 está verde o rojo.

Alimentación elegida

USB de la computadora → placa UNO → pines digitales → LEDs y resistencias → GND. El condensador cerámico 100 nF va entre 5V y GND, cerca del mazo; no conecta LEDs directamente a 5V.

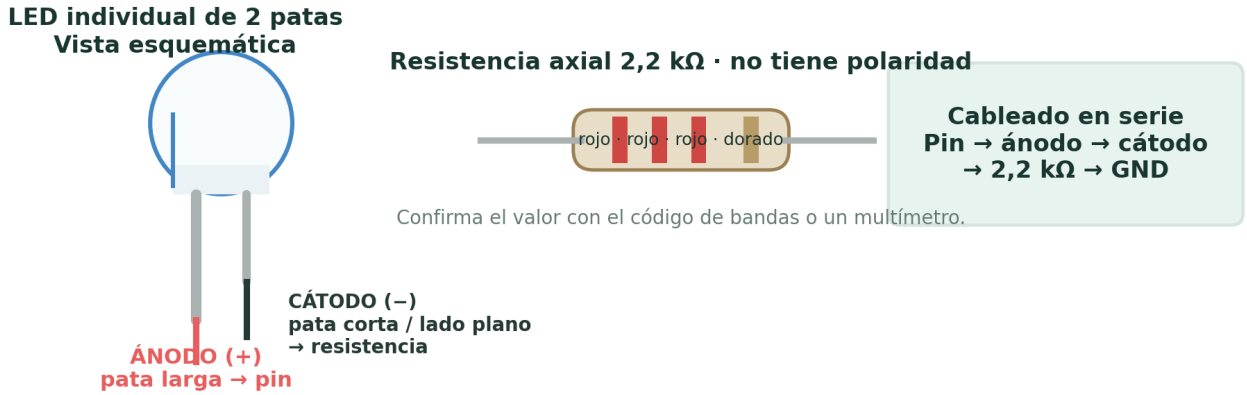
No uses la maqueta como señal vial real

El arranque puede tener un instante con salidas apagadas. Este diseño sirve para una maqueta educativa, no para dirigir tráfico real.

Ánodo, cátodo y resistencia

Las patas largas ayudan si aún no se recortaron; después revisa la forma del LED.

Reconocer el LED, la resistencia y la polaridad



Confirma el valor con el código de bandas o un multímetro.

Si ya recortaste las patas, identifica la marca plana del cuerpo y consulta la hoja de datos. No adivines la polaridad.

- Ánodo (+): normalmente pata larga y electrodo interno más pequeño. Va al cable de señal del Arduino.
- Cátodo (−): normalmente pata corta y lado plano del cuerpo. Va a una resistencia propia de 2,2 kΩ y luego a GND.
- La resistencia no tiene polaridad. En una de 4 bandas y 5 %: rojo-rojo-rojo-dorado indica 2,2 kΩ.
- Si el LED tiene marcas distintas, está recortado o es un módulo RGB, confirma su hoja de datos antes de soldar.

Por qué proponemos 2,2 kΩ

Sin la ficha de tus LEDs no se puede afirmar su caída de tensión exacta.

Fórmula de un LED en serie

R = (Vfuente − Vf del LED) / I. Ejemplo rojo: (5−2)/2200 ≈ 1,36 mA. Potencia: P≈I²R≈4 mW, por debajo de una resistencia común de ¼ W.

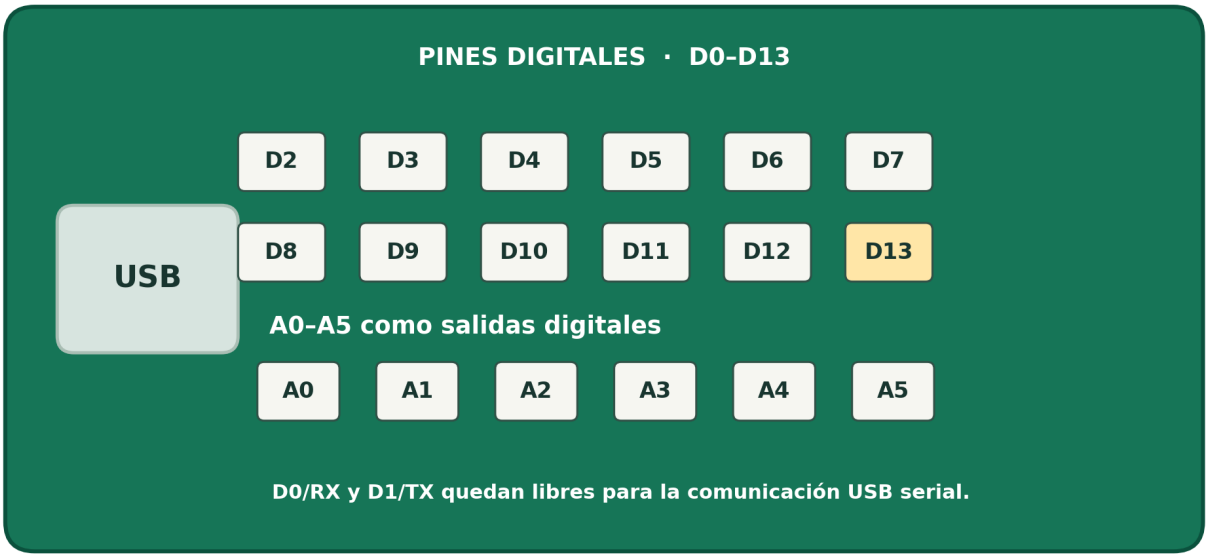
Ejemplo, no medido	Cálculo	Corriente
Rojo · Vf ejemplo 2,0 V	(5−2,0)/2200	1,36 mA
Amarillo · Vf ejemplo 2,1 V	(5−2,1)/2200	1,32 mA
Verde · Vf ejemplo 2,2 V	(5−2,2)/2200	1,27 mA

- Estas corrientes son bajas y reducen la carga; el brillo depende de cada LED. 2,2 kΩ, ¼ W es la elección conservadora descrita en el proyecto.
- 20 mA es una corriente de trabajo recomendada máxima por pin en el UNO, no una meta. 40 mA es un límite absoluto del microcontrolador: no diseñes para acercarte.
- Con seis LEDs encendidos a la vez y alrededor de 1,3 mA cada uno, la carga estimada es cerca de 8 mA, más el indicador integrado de D13.
- Sin ficha técnica no se puede cerrar el cálculo del LED real: necesitas Vf y la corriente indicada. Si el brillo queda bajo, no reduzcas R sin esa ficha. No uses tiras ni LEDs de potencia directamente en los pines.

Pines que aparecen en la tabla

El dibujo agrupa conectores para que ubiques las etiquetas; no está a escala.

Arduino UNO R3 · pines usados por el firmware



Dibujo funcional para localizar nombres de pines; no está a escala ni sustituye la serigrafía de tu placa.

Semáforo	Rojo (+)	Amarillo (+)	Verde (+)
S1 / I1	D2	D3	D4
S2 / I2	D5	D6	D7
S3 / I3	D8	D9	D10
S4 / I4	D11	D12	D13
S5 / I5	A0	A1	A2
S6 / I6	A3	A4	A5

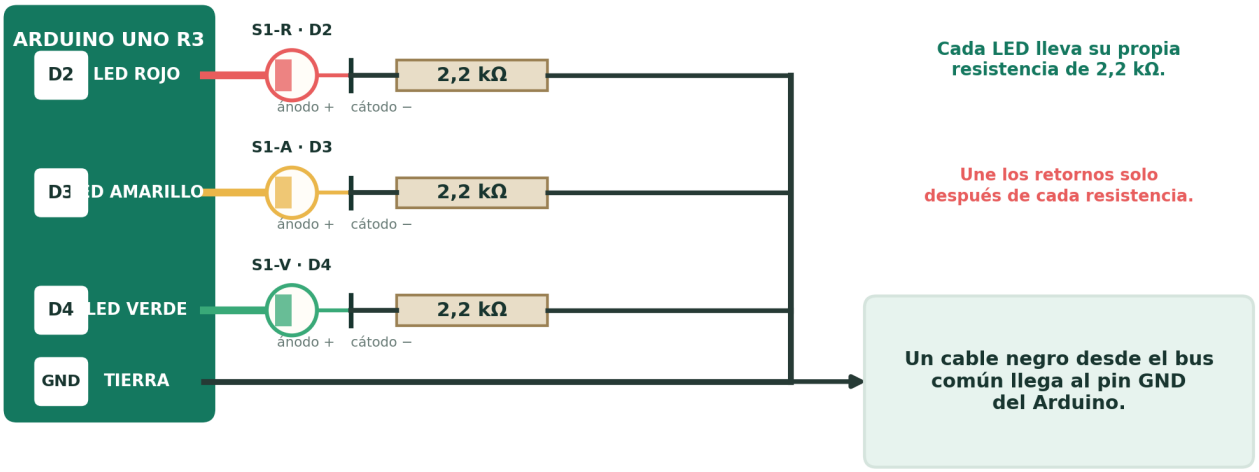
D0/RX y D1/TX quedan para USB serial. A0–A5 funcionan como salidas digitales en el UNO R3. Cada destino de la tabla es el ánodo (+) del LED.

La ruta completa de una luz

Ejemplo: rojo de S1. Las otras dos luces repiten la misma ruta con D3 y D4.

Ejemplo real de cableado · semáforo S1

Tres LEDs separados · tres resistencias individuales · un retorno GND compartido

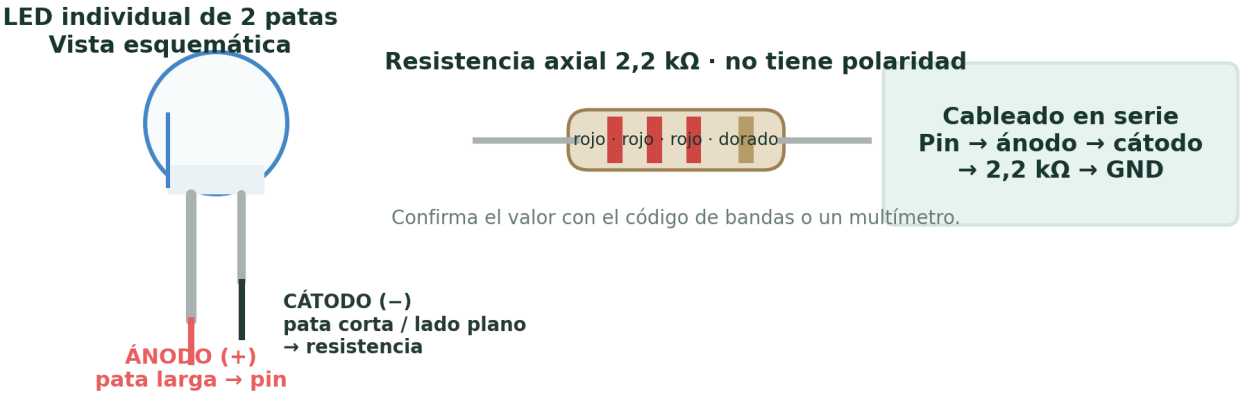


- Señal roja S1: pin D2 → cable rojo → ánodo largo del LED rojo.
- Desde el cátodo corto: resistencia individual 2,2 kΩ → cable/bus negro → pin GND del Arduino.
- S1 amarillo usa D3 y su propio resistor; S1 verde usa D4 y su propio resistor.
- El GND común se forma después de cada resistor. No compartas una resistencia entre LEDs ni unas los cátodos antes de ellas.

Paso 1 · ordenar los tres LEDs

Haz una sola unidad completa y pruébala antes de repetirla seis veces.

Reconocer el LED, la resistencia y la polaridad



Confirma el valor con el código de bandas o un multímetro.

Si ya recortaste las patas, identifica la marca plana del cuerpo y consulta la hoja de datos. No adivines la polaridad.

- Prepara un soporte con tres orificios en columna: rojo arriba, amarillo al medio y verde abajo.
- Inserta cada LED desde el frente y separa sus patas por detrás para que no se toquen.
- Antes de cortar, identifica ánodo y cátodo. Deja suficiente metal para soldar; no cortes al ras del encapsulado.
- Si ya no distingues las patas, usa la cara plana/estructura interna o el modo de diodo del multímetro. Marca A (+) y K (−) antes de seguir.

Vista del frente

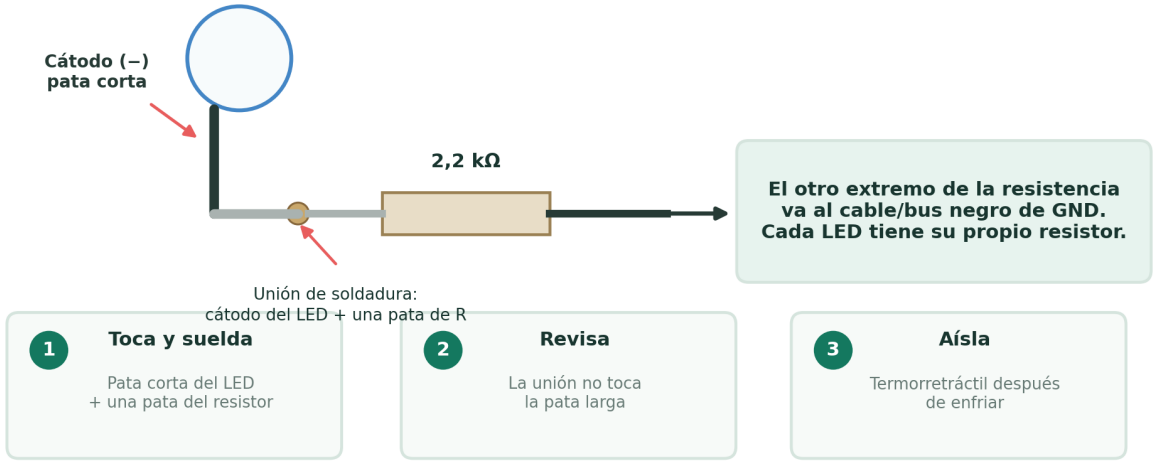
Orden de colores: ROJO · AMARILLO · VERDE. La orientación de las patas se trabaja desde la parte trasera del soporte.

Paso 2 · una resistencia en cada cátodo

Haz las tres uniones con el cable USB desconectado.

Una unión correcta · cátodo → resistor → retorno común

Suelda con el USB desconectado. Deja enfriar cada unión antes de aislarla.



- En el LED rojo, junta solo su pata corta/cátodo con una pata de una resistencia de 2,2 kΩ.
- Repite por separado con el LED amarillo y el verde. Obtendrás tres resistencias, una para cada LED.
- El otro extremo de cada resistencia todavía queda libre: luego se conectará al retorno común negro.
- Toca la unión con el cautín el tiempo mínimo necesario; deja enfriar antes de mover. No calientes el cuerpo plástico del LED.

Paso 3 · llevar cada señal al ánodo

Los tres cables de señal salen del Arduino y terminan en los ánodos largos.

Luz	Sale de	Llega a	Cable	Etiqueta en ambos extremos
LED rojo	Ánodo (+)	D2	Rojo	S1-R / D2
LED amarillo	Ánodo (+)	D3	Amarillo	S1-A / D3
LED verde	Ánodo (+)	D4	Verde	S1-V / D4

- Suelda un cable rojo al ánodo del LED rojo; el extremo opuesto llegará al pin D2.
- Suelda un cable amarillo al ánodo del amarillo; el otro extremo llegará a D3.
- Suelda un cable verde al ánodo del verde; el otro extremo llegará a D4.
- Antes de cerrar el soporte, marca cada cable por color y por ambos extremos. No cruces señales aunque los cables compartan el mismo color entre semáforos.

Recuento de cables de este semáforo

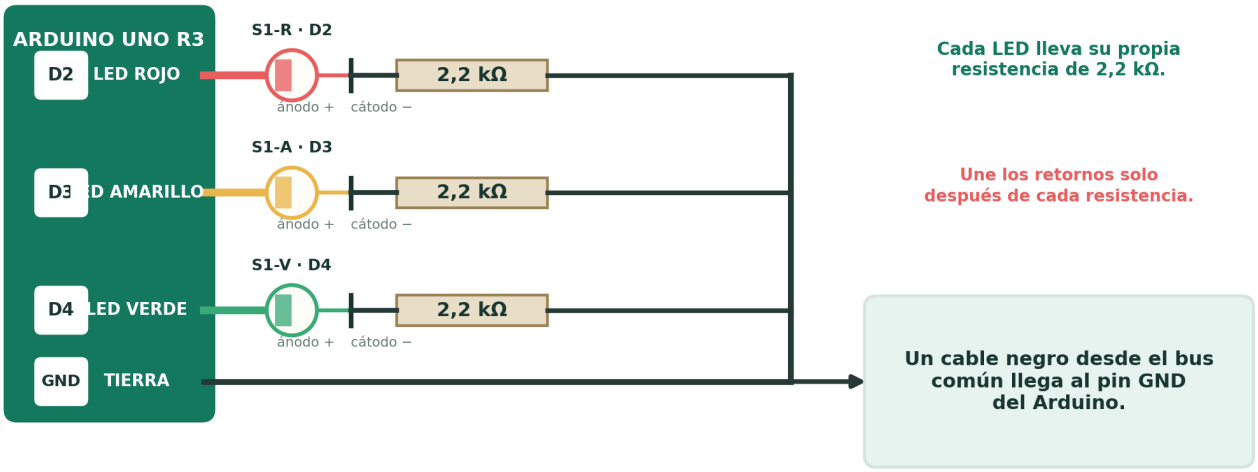
Salen cuatro conductores: rojo, amarillo, verde y un negro de retorno común. Los retornos de los tres LEDs pasan antes por sus tres resistencias individuales.

Paso 4 · formar el retorno GND

La resistencia pertenece a su LED; solo el otro extremo comparte tierra.

Ejemplo real de cableado · semáforo S1

Tres LEDs separados · tres resistencias individuales · un retorno GND compartido



- Suelda el extremo libre del resistor rojo a una zona expuesta del conductor negro común.
- Haz lo mismo con el resistor amarillo y luego con el verde. Cada rama debe conservar su resistencia.
- Desde ese conductor negro sale un solo cable hacia cualquiera de los pines GND del UNO.
- Aísla cada ramificación con tubo termorretráctil y fija el mazo al soporte para que un tirón no llegue a las patas del LED.

Condensador de desacoplo

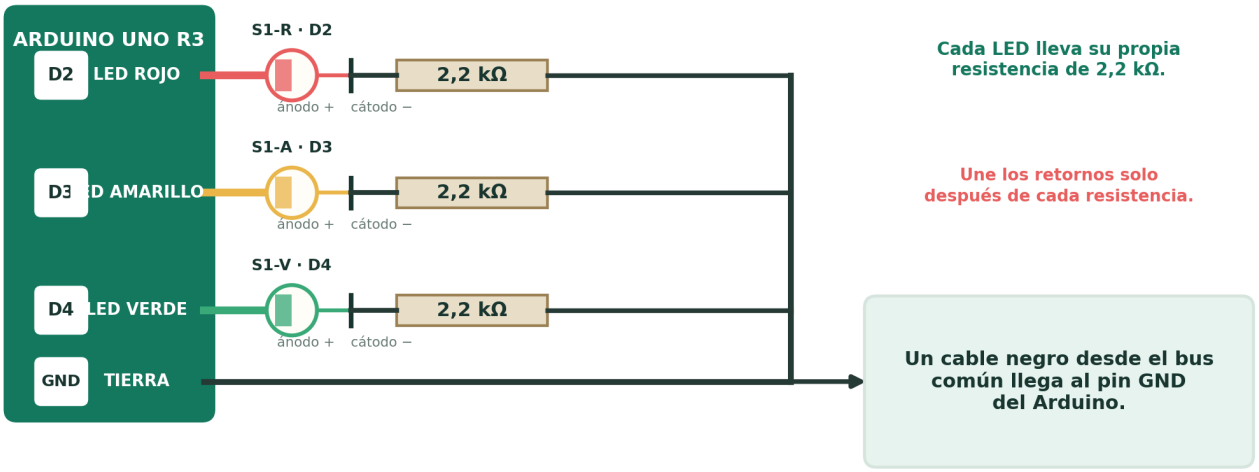
El cerámico de 100 nF se conecta entre 5V y GND cerca de la distribución. No tiene polaridad; no reemplaza las resistencias.

S1 completo: señal y retorno

Este ejemplo repite el circuito que usarás en las seis unidades.

Ejemplo real de cableado · semáforo S1

Tres LEDs separados · tres resistencias individuales · un retorno GND compartido



S	Luz	Sale de	Llega a	Cable	Retorno
S1	Rojo	Ánodo LED rojo	D2	Rojo	Cátodo → R 2,2 kΩ → GND
S1	Amarillo	Ánodo LED amarillo	D3	Amarillo	Cátodo → R 2,2 kΩ → GND
S1	Verde	Ánodo LED verde	D4	Verde	Cátodo → R 2,2 kΩ → GND
S1	Retorno	Extremo libre de las 3 R	GND UNO	Negro	Cada LED conserva su resistor propio

Comprueba cada etiqueta antes de soldar el otro extremo a la placa. El color del conductor identifica la señal, no el número del pin.

Tabla de conexiones · S1, S2 y S3

Las patas largas de los LEDs llegan a los pines indicados; cada cátodo tiene su resistor.

Semáforo	Luz	Sale de	Llega a	Color	Retorno del cátodo (−)
S1 / I1	Rojo	Ánodo (+) del LED rojo	D2	Rojo	Cátodo (−) → 2,2 kΩ propio → GND común
S1 / I1	Amarillo	Ánodo (+) del LED amarillo	D3	Amarillo	Cátodo (−) → 2,2 kΩ propio → GND común
S1 / I1	Verde	Ánodo (+) del LED verde	D4	Verde	Cátodo (−) → 2,2 kΩ propio → GND común
S2 / I2	Rojo	Ánodo (+) del LED rojo	D5	Rojo	Cátodo (−) → 2,2 kΩ propio → GND común
S2 / I2	Amarillo	Ánodo (+) del LED amarillo	D6	Amarillo	Cátodo (−) → 2,2 kΩ propio → GND común
S2 / I2	Verde	Ánodo (+) del LED verde	D7	Verde	Cátodo (−) → 2,2 kΩ propio → GND común
S3 / I3	Rojo	Ánodo (+) del LED rojo	D8	Rojo	Cátodo (−) → 2,2 kΩ propio → GND común
S3 / I3	Amarillo	Ánodo (+) del LED amarillo	D9	Amarillo	Cátodo (−) → 2,2 kΩ propio → GND común
S3 / I3	Verde	Ánodo (+) del LED verde	D10	Verde	Cátodo (−) → 2,2 kΩ propio → GND común

- S1 repite el ejemplo de la página anterior; S2 y S3 usan su propia terna de pines.
- Para cada fila, suelda el cable de color al ánodo. La columna Retorno describe una segunda ruta desde el cátodo a través de una resistencia propia.
- No conectes los 9 LEDs de este grupo a la vez: son salidas separadas y el firmware las gobierna por fase.

Tabla de conexiones · S4, S5 y S6

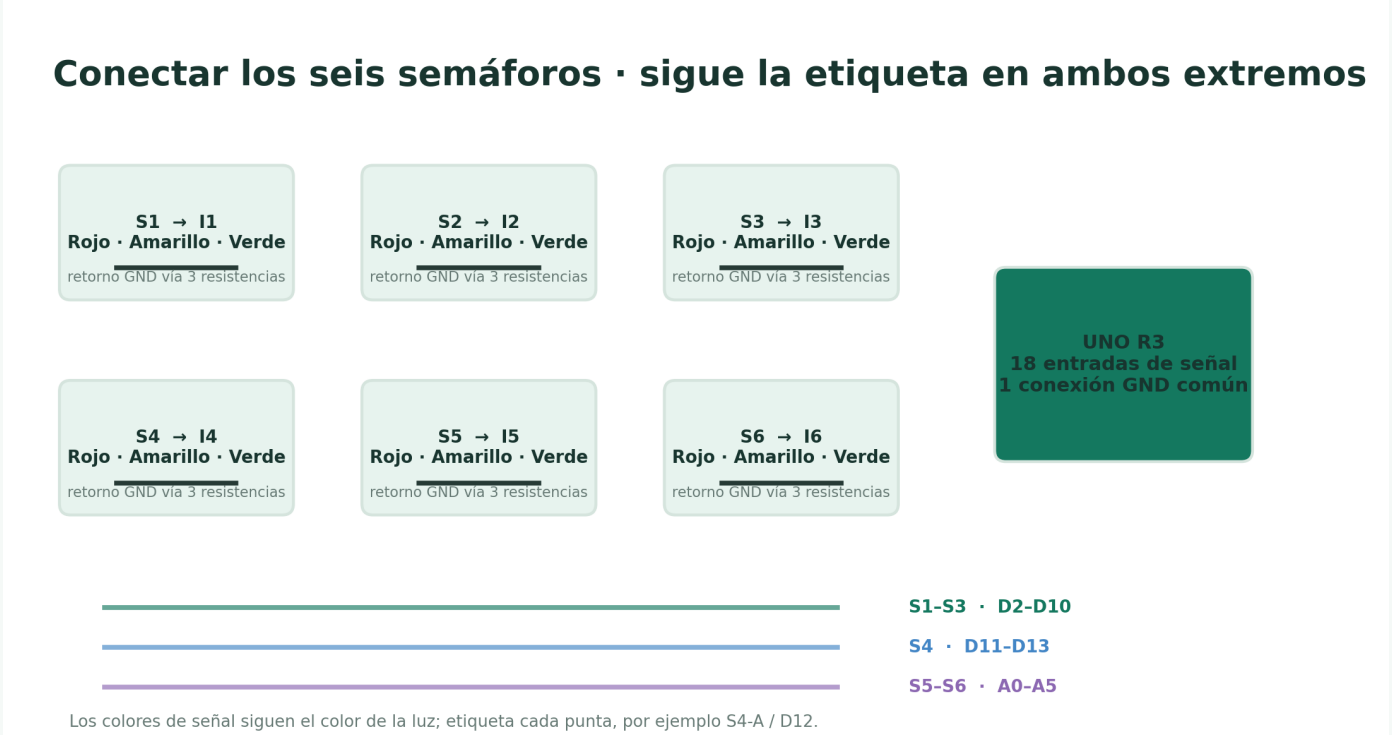
Los pines analógicos se usan como digitales; D0 y D1 siguen reservados al USB serial.

Semáforo	Luz	Sale de	Llega a	Color	Retorno del cátodo (−)
S4 / I4	Rojo	Ánodo (+) del LED rojo	D11	Rojo	Cátodo (−) → 2,2 kΩ propio → GND común
S4 / I4	Amarillo	Ánodo (+) del LED amarillo	D12	Amarillo	Cátodo (−) → 2,2 kΩ propio → GND común
S4 / I4	Verde	Ánodo (+) del LED verde	D13	Verde	Cátodo (−) → 2,2 kΩ propio → GND común
S5 / I5	Rojo	Ánodo (+) del LED rojo	A0	Rojo	Cátodo (−) → 2,2 kΩ propio → GND común
S5 / I5	Amarillo	Ánodo (+) del LED amarillo	A1	Amarillo	Cátodo (−) → 2,2 kΩ propio → GND común
S5 / I5	Verde	Ánodo (+) del LED verde	A2	Verde	Cátodo (−) → 2,2 kΩ propio → GND común
S6 / I6	Rojo	Ánodo (+) del LED rojo	A3	Rojo	Cátodo (−) → 2,2 kΩ propio → GND común
S6 / I6	Amarillo	Ánodo (+) del LED amarillo	A4	Amarillo	Cátodo (−) → 2,2 kΩ propio → GND común
S6 / I6	Verde	Ánodo (+) del LED verde	A5	Verde	Cátodo (−) → 2,2 kΩ propio → GND común

- S4 usa D11, D12 y D13. D13 comparte función con el LED indicador integrado del UNO.
- S5 usa A0–A2 y S6 usa A3–A5; en Arduino IDE se llaman A0…A5 aunque funcionen como digitales.
- Hay 18 salidas físicas en total y ninguna sobra para sensores. Los vehículos y cierres de la aplicación son simulación.

Agrupa y etiqueta los cables

La posición física exacta del Arduino en la base depende de tu maqueta.



Los colores de señal siguen el color de la luz; etiqueta cada punta, por ejemplo S4-A / D12.

Etiqueta cada punta con semáforo, luz y pin

Etiqueta ejemplo	Identifica	Color conductor
S1-R / D2	Rojo S1	cable rojo
S2-A / D6	Amarillo S2	cable amarillo
S4-V / D13	Verde S4	cable verde
S5-R / A0	Rojo S5	cable rojo
S6-V / A5	Verde S6	cable verde
TODOS-GND	Retornos después de resistores	negro

Sujeta los mazos al tablero con una abrazadera o fijación aislante; deja un pequeño alivio de tensión en cada semáforo.

Lista de comprobación eléctrica

Haz cualquier modificación con la alimentación desconectada.

- [ ] Hay 18 LEDs individuales de dos patas, colocados rojo/amarillo/verde.
- [ ] Cada ánodo está en el pin asignado por la tabla; cada cátodo tiene una resistencia de 2,2 kΩ.
- [ ] Ningún resistor se comparte entre LEDs. Solo los extremos posteriores de los resistores se unen a GND.
- [ ] Todos los empalmes están aislados; no queda cobre de señales tocando otro cable o la base metálica.
- [ ] El cable negro de retorno termina en un pin GND del UNO. No se conectó por error a 5V.
- [ ] El condensador cerámico 100 nF, si se instala, va entre 5V y GND y no tiene polaridad.
- [ ] No hay cortocircuito entre las señales y GND. Con multímetro, revisa continuidad y valor de resistencias con el USB quitado.
- [ ] El cable USB alimentará solo el UNO y los LEDs de baja corriente descritos; no hay tiras ni lámparas de potencia.
- [ ] D0/RX y D1/TX no se usaron para las luces. La serigrafía de la placa coincide con el pin de cada etiqueta.
- [ ] El firmware aún tiene MAPA_CONFIRMADO=false durante la revisión inicial.

Si una soldadura está mal

No la corrijas con el Arduino conectado. Desconecta el USB, deja enfriar el cautín, corrige, aísla y vuelve a revisar con multímetro.

Cargar el programa del proyecto

Código fuente: arduino/SmartTraffic/SmartTraffic.ino. El apéndice de este PDF copia el mismo archivo.

- Instala Arduino IDE y conecta el UNO con un cable USB de datos. Comprueba que el sistema operativo detecte el puerto.
- En Arduino IDE, abre el archivo SmartTraffic.ino y selecciona Herramientas → Placa → Arduino AVR Boards → Arduino Uno.
- Selecciona Herramientas → Puerto y el puerto recién conectado. Si no aparece, revisa cable, conector y controlador USB-serial de la placa.
- Pulsa Verificar para compilar. Luego pulsa Subir y espera el mensaje de carga completada.
- Con MAPA_CONFIRMADO=false, las seis luces rojas se encienden y la secuencia automática permanece bloqueada.
- Abre el Monitor serie a 115200 baudios, con final de línea Nueva línea. Debe aparecer INICIO,1 y estados BLOQUEADO.
- Cierra el Monitor serie antes de abrir el puente del sistema web; el puerto serial admite un solo programa a la vez.

No habilites aún la secuencia

MAPA_CONFIRMADO está desactivado intencionalmente. Prueba los 18 LEDs y revisa en la maqueta que S1–S6 correspondan a I1–I6 antes de activarlo.

Probar cada color por separado

El comando dura 2 segundos; envía una línea por vez en el Monitor serie.

Formato del comando

TEST,<número de semáforo>,<color> · color: R (rojo), A (amarillo), V (verde). Ejemplo: TEST,1,R

Unidad	Escribe en líneas separadas	Cruce / pines
S1	TEST,1,R · TEST,1,A · TEST,1,V	I1 · D2–D4
S2	TEST,2,R · TEST,2,A · TEST,2,V	I2 · D5–D7
S3	TEST,3,R · TEST,3,A · TEST,3,V	I3 · D8–D10
S4	TEST,4,R · TEST,4,A · TEST,4,V	I4 · D11–D13
S5	TEST,5,R · TEST,5,A · TEST,5,V	I5 · A0–A2
S6	TEST,6,R · TEST,6,A · TEST,6,V	I6 · A3–A5

- Para cada S, envía primero R, espera dos segundos y confirma el rojo; repite con A y luego V.
- Durante la prueba, los semáforos no seleccionados quedan en rojo. Deja la maqueta inmóvil y libre de vehículos.
- Si se enciende otro color, comprueba el cable y pin con la tabla. Desconecta el USB antes de cambiar soldaduras.
- Cuando termines, el firmware vuelve a rojo bloqueado. Repite cualquier prueba que no sea clara.

Habilitar la secuencia solo después de verificar

Este paso activa seis salidas físicas a la vez según el eje horizontal.

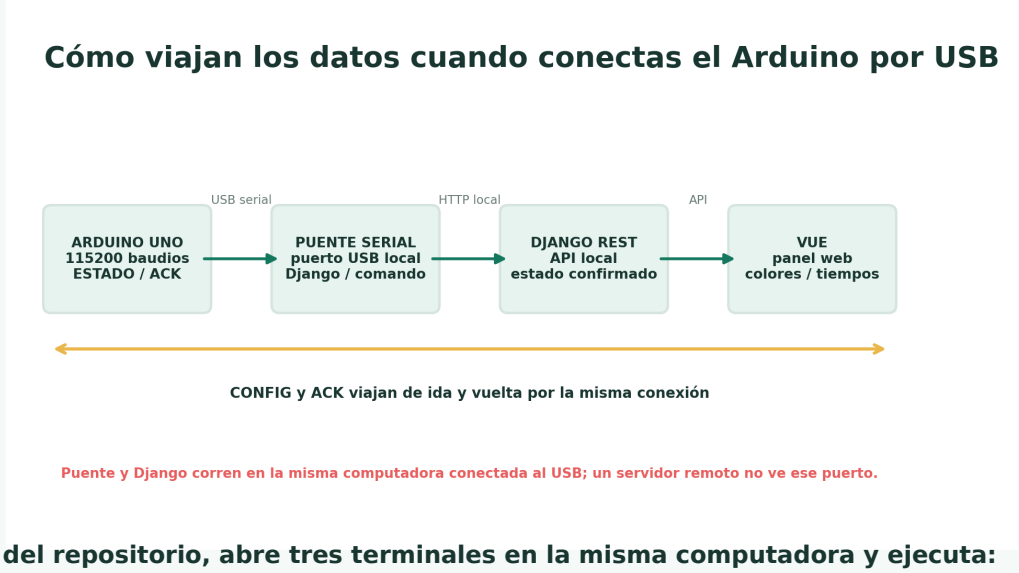
- Confirma que cada semáforo está en el cruce propuesto: S1→I1, S2→I2, S3→I3, S4→I4, S5→I5, S6→I6.
- Orienta las caras hacia los vehículos que usan las vías horizontales. En la vía perpendicular instala una señal de ceda el paso.
- Revisa que las flechas de la maqueta siguen las direcciones confirmadas: horizontal superior ←, horizontal inferior →; verticales ↑, ↓, ↑.
- En el código, cambia const bool MAPA_CONFIRMADO = false; a true. Mantén VIA_SIN_SEMAFORO_CEDE = true.
- Verifica CRUCE = {1,2,3,4,5,6} y EJE = {'H','H','H','H','H','H'}. Si cambias la orientación de un semáforo, revisa su EJE y la seguridad del cruce.
- Vuelve a cargar el sketch. El ciclo concede más verde a la horizontal y usa amarillo y todo-rojo antes de cambiar de eje.
- Como los seis EJE son H, en la fase secundaria V no se enciende ninguna luz: los seis semáforos quedan rojos. El ceda es una señal pasiva, no un verde controlado por el Arduino.

No cambies esta constante por probar

Si todavía no has confirmado montaje, dirección de cada cara y señales de ceda, deja MAPA_CONFIRMADO=false: todos quedarán en rojo.

Conectar el UNO a SmartTraffic

El USB se lee con un puente serial local; un servidor remoto no puede acceder al puerto de tu computadora.



Desde la raíz del repositorio, abre tres terminales en la misma computadora y ejecuta:

1 · Backend

```
cd backend
./env/bin/python manage.py migrate
./env/bin/python manage.py runserver 127.0.0.1:8000
```

2 · Puente USB

```
cd backend
./env/bin/python manage.py puente_serial --listar
./env/bin/python manage.py puente_serial --puerto /dev/ttyACM0
```

3 · Frontend

```
cd frontend
npm install
npm run dev -- --host 127.0.0.1
```

En macOS el puerto suele ser /dev/cu.usbmodem...; en Windows, COM3 (reemplaza el ejemplo). En Windows usa env\Scripts\python.exe si creaste allí el entorno virtual. Vite abre http://127.0.0.1:5173. Cierra el Monitor serie antes de iniciar el puente.

Qué ver en la aplicación

La interfaz diferencia estados confirmados por Arduino de valores que solo pertenecen a la simulación.

- Si el puente abre el puerto, Django recibe ESTADO y la web marca la fuente Arduino. La API de estado es /api/dispositivo/.
- Mientras MAPA_CONFIRMADO=false, el dispositivo informa BLOQUEADO. La comunicación funciona, pero la secuencia física no arranca.
- Con EJE={H,H,H,H,H,H}, las seis luces horizontales comparten verde/amarillo. En la fase reservada a V las seis permanecen rojas; no existe un verde físico para la vía secundaria.
- Si no llegan mensajes durante 4 s, Django marca desconexión y la interfaz limpia los datos viejos; no muestra colores antiguos como actuales.
- El reporte serial refleja lo que ordena el programa; no mide ópticamente el LED ni comprueba el cableado.
- Los vehículos, cierres y velocidad del panel son simulados. Sin sensores físicos, el tráfico simulado no modifica el Arduino automáticamente.

Acción	Mensaje serial
Solicitud web	CONFIG,17,30,15,3,2
Respuesta aplicada	ACK,17,APLICADO,OK
Respuesta rechazada	ACK,17,RECHAZADO,VALORES

La configuración se considera aplicada en la web después del ACK del firmware. Con el mapa bloqueado, el ACK no arranca el ciclo.

Fallos de luces y cableado

Desconecta USB antes de tocar conductores o rehacer una soldadura.

Síntoma	Causa posible	Qué comprobar
Un LED no enciende	Polaridad invertida, resistor/empalme abierto, pin equivocado o soldadura fría.	Desconecta USB; revisa ánodo, cátodo, resistor y etiqueta. Prueba ese LED con TEST,s,color.
Se enciende otro color	Los cables de señal intercambiaron D2/D3/D4 (o la terna de otro S).	Sigue el rótulo desde el ánodo al pin de la tabla. Corrige sin alimentación y repite el test.
No funciona todo un semáforo	Retorno GND abierto, cuatro cables sin continuidad o S conectado a otra terna.	Mide los tres caminos y el bus común; confirma que el cable negro llega a GND del UNO.
Varias señales inesperadas	Etiquetas repetidas, retornos unidos antes de resistores o cruce de mazos.	Revisa que cada LED tenga su resistor y que S1–S6 usen grupos distintos de tres pines.

Si huele a quemado, un pin se calienta o ves un cable pelado, desconecta USB de inmediato y no continúes hasta corregir la causa.

Fallos de USB, carga o aplicación

Comprueba primero que solo haya un programa abierto sobre el puerto serial.

Síntoma	Causa posible	Qué comprobar
La PC no ve el UNO	Cable USB solo de carga, puerto físico o controlador USB-serial.	Prueba cable de datos y otro puerto; revisa Administrador de dispositivos o lista de puertos del IDE.
Arduino IDE no carga	Placa/puerto incorrectos, cable o puerto ocupado por Monitor serie/puente.	Cierra los otros programas seriales, selecciona Arduino Uno y el puerto correcto, compila y sube otra vez.
La página no recibe datos	Puente no iniciado, puerto equivocado o Django no está activo.	Ejecuta --listar, inicia puente_serial, revisa la terminal del backend y visita /api/dispositivo/.
Tiempo no aplicado	Mapa bloqueado, valor fuera de rango, puerto desconectado o rechazo/timeout.	Lee el aviso y ACK; revisa principal 15–120 s, secundaria 10–90 s y principal > secundaria.

- Amarillo permitido: 3–10 s. Intervalo todo-rojo: 2–10 s.
- El puerto puede ser /dev/ttyACM0 en Linux, /dev/cu.usbmodem… en macOS o COM3 en Windows.
- El Monitor serie, Arduino IDE y el puente no pueden compartir simultáneamente el mismo puerto.

Qué se verificó y qué falta probar

Esta guía se basó en el firmware y las tablas que están dentro del repositorio.

Verificado en el código y la documentación

- 18 salidas directas, orden rojo/amarillo/verde, D0/D1 reservados y pines exactos S1–S6.
- Retorno individual a través de 18 resistencias, común GND, USB serial a 115200 y protocolo INICIO/ESTADO/CONFIG/ACK/TEST.
- Inicio bloqueado con todos los rojos y la constante MAPA_CONFIRMADO=false; tests automatizados de protocolo/firmware ya descritos en docs/PRUEBAS.md.

Suposiciones o pruebas físicas pendientes

- Las fotografías muestran puntos de ubicación, pero no confirman la cara exacta de cada semáforo. La guía propone una luz por cruce mirando a la horizontal; confirma esto en la maqueta.
- No se proporcionó la hoja de datos ni una foto de tus LEDs/resistencias. El cálculo con Vf es un ejemplo; verifica que tus LEDs sean indicadores de baja corriente de dos patas y confirma 2,2 kΩ.
- No hubo Arduino conectado ni se midieron corrientes, soldaduras, caída de tensión, brillo, detección de puerto o ACK con hardware real.
- Durante la fase secundaria del programa, las salidas quedan en rojo porque los seis semáforos están asignados al eje H; no hay una luz verde vertical física.
- El firmware se compiló y probó en entorno simulado/documentado; las fotos de circuito y el mapa no son una inspección física de tu placa.

Dibujo de portada e imágenes

La portada fue generada como ilustración basada en la maqueta. Los pines y rutas eléctricas se dibujaron como esquemas vectoriales y se cotejaron con el firmware; no se usan imágenes generadas para afirmar conexiones.

SmartTraffic.ino · página 1 de 3

Texto tomado del archivo arduino/SmartTraffic/SmartTraffic.ino; no es otro firmware.

Líneas del sketch; conserva la indentación al copiarlo al Arduino IDE.

```
001  // SmartTraffic para Arduino UNO R3 (ATmega328P). Protocolo ASCII a 115200 baudios.
002  // El mapa físico se entrega DESACTIVADO hasta confirmar cada semaforo y su vía.
003  const bool MAPA_CONFIRMADO = false;
004  const bool VIA_SIN_SEMAFORO_CEDE = true; // Confirmado por el usuario para la maqueta.
005  const byte CRUCE[6] = {1, 2, 3, 4, 5, 6}; // S1–S6 propuestos para I1–I6.
006  const char EJE[6] = {'H', 'H', 'H', 'H', 'H', 'H'}; // H = horizontal; V = vertical.
007
008  // Por semaforo: rojo, amarillo, verde. D0/D1 se reservan para USB serial.
009  const byte PINES[6][3] = {
010    {2, 3, 4}, {5, 6, 7}, {8, 9, 10}, {11, 12, 13},
011    {A0, A1, A2}, {A3, A4, A5}
012  };
013
014  enum Color { ROJO, AMARILLO, VERDE };
015  enum Fase { ROJO_A_H, H_VERDE, H_AMARILLO, ROJO_A_V, V_VERDE, V_AMARILLO };
016  struct Tiempos { unsigned int principal, secundaria, amarillo, todoRojo; };
017  Tiempos tiempos = {30, 15, 3, 2};
018  Tiempos pendientes;
019  Fase fase = ROJO_A_H;
020  Color colores[6] = {ROJO, ROJO, ROJO, ROJO, ROJO, ROJO};
021  unsigned long inicioFase = 0, ultimoReporte = 0, inicioPrueba = 0, idPendiente = 0;
022  int semaforoPrueba = -1;
023  Color colorPrueba = ROJO;
024  bool hayPendiente = false;
025  char linea[96];
026  byte longitud = 0;
027  bool descartarLinea = false;
028
029  bool mapaValido() {
030    if (!MAPA_CONFIRMADO || !VIA_SIN_SEMAFORO_CEDE) return false;
031    for (byte i = 0; i < 6; i++) {
032      if (CRUCE[i] < 1 || CRUCE[i] > 6 || (EJE[i] != 'H' && EJE[i] != 'V')) return false;
033    }
034    return true;
035  }
036
037  unsigned long duracionFase() {
038    if (fase == H_VERDE) return (unsigned long)tiempos.principal * 1000UL;
039    if (fase == V_VERDE) return (unsigned long)tiempos.secundaria * 1000UL;
040    if (fase == H_AMARILLO || fase == V_AMARILLO) return (unsigned long)tiempos.amarillo * 1000UL;
041    return (unsigned long)tiempos.todoRojo * 1000UL;
042  }
043
044  void avanzarFase(unsigned long ahora) {
045    if (ahora - inicioFase < duracionFase()) return;
046    inicioFase = ahora;
047    switch (fase) {
048      case ROJO_A_H: fase = H_VERDE; break;
049      case H_VERDE: fase = H_AMARILLO; break;
050      case H_AMARILLO: fase = ROJO_A_V; break;
051      case ROJO_A_V: fase = V_VERDE; break;
052      case V_VERDE: fase = V_AMARILLO; break;
053      case V_AMARILLO: fase = ROJO_A_H; break;
054    }
055  }
056
057  void confirmar(unsigned long id, const char *resultado, const char *mensaje) {
058    Serial.print(F("ACK,")); Serial.print(id); Serial.print(',');
059    Serial.print(resultado); Serial.print(','); Serial.println(mensaje);
060  }
061
062  bool leerNumero(const char *texto, unsigned long &valor) {
063    if (!texto || !*texto) return false;
064    valor = 0;
065    while (*texto) {
066      if (*texto < '0' || *texto > '9') return false;
067      byte digito = *texto++ - '0';
068      if (valor > (4294967295UL - digito) / 10UL) return false;
069      valor = valor * 10UL + digito;
070    }
071    return true;
072  }
073
074  void procesarConfiguracion(char *partes[], byte cantidad) {
075    unsigned long valores[5];
076    if (cantidad != 6 || !leerNumero(partes[1], valores[0]) || valores[0] == 0) return;
077    for (byte i = 1; i < 5; i++) {
078      if (!leerNumero(partes[i + 1], valores[i])) { confirmar(valores[0], "RECHAZADO", "FORMATO"); return; }
079    }
080    if (valores[1] < 15 || valores[1] > 120 || valores[2] < 10 || valores[2] > 90 ||
081        valores[1] <= valores[2] || valores[3] < 3 || valores[3] > 10 || valores[4] < 2 || valores[4] > 10) {
082      confirmar(valores[0], "RECHAZADO", "VALORES"); return;
```

SmartTraffic.ino · página 2 de 3

Texto tomado del archivo arduino/SmartTraffic/SmartTraffic.ino; no es otro firmware.

Líneas del sketch; conserva la indentación al copiarlo al Arduino IDE.

```
083    }
084    if (hayPendiente) { confirmar(valores[0], "RECHAZADO", "OCUPADO"); return; }
085    pendientes = {(unsigned int)valores[1], (unsigned int)valores[2], (unsigned int)valores[3], (unsigned
        int)valores[4]};
086    idPendiente = valores[0];
087    hayPendiente = true;
088  }
089
090  void procesarPrueba(char *partes[], byte cantidad) {
091    unsigned long semaforo;
092    if (mapaValido() || cantidad != 3 || !leerNumero(partes[1], semaforo) || semaforo < 1 || semaforo > 6)
        return;
093    if (partes[2][1] != '\0') return;
094    if (partes[2][0] == 'R') colorPrueba = ROJO;
095    else if (partes[2][0] == 'A') colorPrueba = AMARILLO;
096    else if (partes[2][0] == 'V') colorPrueba = VERDE;
097    else return;
098    semaforoPrueba = (int)semaforo - 1;
099    inicioPrueba = millis();
100    Serial.print(F("PRUEBA,")); Serial.print(semaforo); Serial.print(','); Serial.println(partes[2]);
101  }
102
103  void procesarLinea() {
104    char *partes[7];
105    byte cantidad = 0;
106    char *parte = strtok(linea, ",");
107    while (parte && cantidad < 7) { partes[cantidad++] = parte; parte = strtok(NULL, ","); }
108    if (cantidad && strcmp(partes[0], "CONFIG") == 0) procesarConfiguracion(partes, cantidad);
109    else if (cantidad && strcmp(partes[0], "TEST") == 0) procesarPrueba(partes, cantidad);
110  }
111
112  void leerSerial() {
113    while (Serial.available()) {
114      char c = (char)Serial.read();
115      if (c == '\r') continue;
116      if (c == '\n') {
117        if (!descartarLinea && longitud) { linea[longitud] = '\0'; procesarLinea(); }
118        longitud = 0; descartarLinea = false;
119      } else if (!descartarLinea) {
120        if (longitud < sizeof(linea) - 1) linea[longitud++] = c;
121        else { longitud = 0; descartarLinea = true; }
122      }
123    }
124  }
125
126  void aplicarConfiguracion(unsigned long ahora) {
127    if (!hayPendiente) return;
128    if (mapaValido() && fase != ROJO_A_H && fase != ROJO_A_V) return;
129    tiempos = pendientes;
130    inicioFase = ahora; // Mantiene completo el intervalo de todo rojo.
131    hayPendiente = false;
132    confirmar(idPendiente, "APLICADO", "OK");
133  }
134
135  void actualizarLuces() {
136    Color deseados[6];
137    bool cambia = false;
138    for (byte i = 0; i < 6; i++) {
139      deseados[i] = ROJO;
140      if (!mapaValido() && semaforoPrueba == i) deseados[i] = colorPrueba;
141      if (mapaValido() && EJE[i] == 'H' && fase == H_VERDE) deseados[i] = VERDE;
142      if (mapaValido() && EJE[i] == 'H' && fase == H_AMARILLO) deseados[i] = AMARILLO;
143      if (mapaValido() && EJE[i] == 'V' && fase == V_VERDE) deseados[i] = VERDE;
144      if (mapaValido() && EJE[i] == 'V' && fase == V_AMARILLO) deseados[i] = AMARILLO;
145      if (deseados[i] != colores[i]) cambia = true;
146    }
147    if (!cambia) return;
148    // Apaga primero todos los verdes y amarillos antes de encender una nueva fase.
149    for (byte i = 0; i < 6; i++) {
150      digitalWrite(PINES[i][VERDE], LOW); digitalWrite(PINES[i][AMARILLO], LOW); digitalWrite(PINES[i][ROJO],
        HIGH);
151    }
152    for (byte i = 0; i < 6; i++) {
153      if (deseados[i] != ROJO) {
154        digitalWrite(PINES[i][ROJO], LOW); digitalWrite(PINES[i][deseados[i]], HIGH);
155      }
156      colores[i] = deseados[i];
157    }
158  }
159
160  void enviarEstado() {
161    Serial.print(F("ESTADO,"));
```

SmartTraffic.ino · página 3 de 3

Texto tomado del archivo arduino/SmartTraffic/SmartTraffic.ino; no es otro firmware.

Líneas del sketch; conserva la indentación al copiarlo al Arduino IDE.

```
  Serial.print(mapaValido() ? F("ACTIVO") : semaforoPrueba >= 0 ? F("PRUEBA") : F("BLOQUEADO"));
  for (byte i = 0; i < 6; i++) {
    Serial.print(','); Serial.print(colores[i] == ROJO ? 'R' : colores[i] == AMARILLO ? 'A' : 'V');
  }
  Serial.println();
}

void setup() {
  for (byte i = 0; i < 6; i++) {
    for (byte c = 0; c < 3; c++) { pinMode(PINES[i][c], OUTPUT); digitalWrite(PINES[i][c], LOW); }
    digitalWrite(PINES[i][ROJO], HIGH);
  }
  Serial.begin(115200);
  inicioFase = millis();
  Serial.println(F("INICIO,1"));
  enviarEstado();
}

void loop() {
  unsigned long ahora = millis();
  leerSerial();
  if (semaforoPrueba >= 0 && ahora - inicioPrueba >= 2000UL) semaforoPrueba = -1;
  if (mapaValido()) avanzarFase(ahora);
  aplicarConfiguracion(ahora);
  actualizarLuces();
  if (ahora - ultimoReporte >= 1000UL) { ultimoReporte = ahora; enviarEstado(); }
}
```